AEGIS

AI Diagnosis + Recovery Planner

A beginner-friendly module guide for Safety-Gated Autonomous Network Recovery

> **Your module is Aegis's reasoning brain: it identifies the most likely cause of a network incident and proposes safe, reversible recovery plans. It does not directly change the real network.**

Hackathon architecture brief

Prepared for the AI Diagnosis + Recovery Planner responsibility

1. The big picture

Aegis is designed to help a network recover from incidents without giving an AI unrestricted control. Your module sits in the middle of the decision process: it turns evidence into an explanation and a proposed recovery plan.

The flow

Stage	Question answered	Result
Monitoring	What is happening?	Metrics, logs, alerts, configuration changes
Incident detection	Is this abnormal?	An incident and its initial scope
Your module	What likely caused it, and what could fix it?	Ranked diagnosis and candidate plans
Digital Twin	What is likely to happen if we try this?	Simulation evidence
Safety Engine	May this exact plan run?	Approval, limits, rejection, or human review
Executor	How is the approved plan applied?	Controlled real-world actions
Verification	Did service actually recover?	Success, rollback, or re-planning

> **Think of it this way: monitors are Aegis's eyes and ears; your module is the doctor; the Digital Twin is the practice patient; the Safety Engine is the cautious supervisor; and the executor is the technician.**

Why the separation matters

An AI can be useful but uncertain. Aegis therefore makes the AI a planner, not a direct operator. Simulation tests predicted consequences, while a rule-based safety layer checks authorization, risk limits, and rollback readiness before the executor can act.

2. What your module should do

A. Diagnose the incident

Diagnosis means moving from visible symptoms to likely causes. A symptom might be “customers cannot reach the payment API.” A root cause might be “a routing change on Router R7 sent payment traffic to the wrong destination.”

• Correlate metrics, logs, topology, service health, and recent changes.

• Build a timeline: what changed first, and what failed immediately afterward?

• Identify affected devices, links, services, users, and regions.

• Generate more than one plausible cause if the evidence is incomplete.

• Give each hypothesis a confidence level and list the evidence for and against it.

• Say when more safe, read-only diagnostics are needed.

B. Plan recovery

Recovery planning turns the diagnosis into explicit steps that can be simulated and checked. A good plan explains the objective, exact actions, preconditions, expected effects, verification checks, stop conditions, and rollback.

• Produce a small set of candidate plans, such as containment, fastest restoration, and root-cause repair.

• Prefer constrained operations from an approved action catalog rather than arbitrary commands.

• Rank plans by predicted success, safety, disruption, blast radius, and rollback difficulty.

• Attach a human-readable explanation for operators and judges.

3. A worked example

Scenario: payment requests in the ap-south region begin failing. The monitoring system finds that a BGP routing configuration changed on Router R7 just before packet loss began. Application servers and DNS are healthy; the backup route through R8 still works.

Diagnosis output

Hypothesis	Confidence	Supporting evidence
Incorrect BGP route on R7	0.82	R7 route change occurred just before loss; backup path works; app and DNS are healthy.
Physical link failure	0.12	Packet loss exists, but link health indicators are normal.
Payment API failure	0.06	Service health checks are passing.

Candidate plans

Plan	Goal	Trade-off
A: Roll back R7 configuration	Restore the last known-good route.	Likely fixes the root cause; brief routing reconvergence is possible.
B: Shift traffic to R8	Restore service using the backup path.	Fast containment, but capacity must be sufficient and root cause remains.
C: Gather more evidence	Run a read-only route lookup and validation.	Lowest risk, but delays recovery if the cause is already clear.

Example plan A

1. Confirm the unexpected route is still present on R7.
2. Save the current configuration.
3. Restore the approved known-good BGP configuration.
4. Validate configuration syntax and routing state.
5. Simulate the change in the Digital Twin.
6. If approved, execute in production.
7. Verify payment connectivity, packet loss, and error rate.
8. Roll back if health worsens or a stop condition is met.

4. Inputs your module receives

Your module should receive a clean incident context package. It should not receive secrets such as passwords, private keys, or access tokens.

Input	Why it matters	Example
Incident metadata	Identifies urgency and initial scope.	INC-1042, critical, payment-api, ap-south
Topology snapshot	Shows dependencies and alternate paths.	Users -> R1 -> R7 -> Payment subnet; R8 is backup
Telemetry	Shows measured network behavior.	Packet loss, latency, CPU, link state, interface errors
Logs and events	Explains what devices and services observed.	BGP updates, firewall denies, configuration errors
Recent changes	Often exposes the triggering event.	A routing-policy update at 14:00:52
Configuration snapshots	Allows comparison and safe rollback.	Current and known-good R7 configuration versions
Runbooks and history	Supplies approved organizational knowledge.	Previous incident fixes and standard procedures
Policies and constraints	Defines what automation may safely attempt.	Max 25% traffic shift; device reboot needs approval

Minimum incident context

At a minimum, include incident ID, timestamps, symptom, current severity, affected service and region, topology version, relevant metrics and events, recent changes, policy version, and the list of approved actions.

5. Outputs your module generates

1. Structured diagnosis report

Machines need predictable fields. The report should include incident summary, affected components, ranked hypotheses, confidence, evidence, conflicting evidence, missing information, and estimated blast radius.

```
{
  "incident_id": "INC-1042",
  "summary": "Payment traffic is failing between R7 and the payment subnet.",
  "hypotheses": [{"cause": "Incorrect BGP route on R7", "confidence": 0.82}],
  "missing_information": ["Post-change BGP validation result"]
}
```

2. Candidate recovery plan

Each plan should include a plan ID, objective, related hypothesis, ordered actions, preconditions, expected impact, risk estimate, verification checks, rollback operation, required permissions, stop conditions, and required human approval.

3. Operator explanation

Also generate a short explanation in plain language: “A routing change on R7 is the strongest root-cause candidate. The recommended plan restores the last known-good configuration. Aegis will simulate it and verify reachability before production execution.”

6. Working with the Digital Twin

The Digital Twin is a safe virtual representation of the network. It models devices, links, routing behavior, firewall policies, traffic flows, capacity, and service dependencies. Your planner sends the full candidate plan to it before any production action.

What the twin tests

• Does connectivity return?

• Does the plan create routing loops, isolation, or unstable convergence?

• Does it overload a backup link or break a different service?

• Does it preserve required redundancy?

• Does the rollback work after a partial or failed action?

Feedback loop

If the planner proposes shifting 100% of traffic to R8 and the Digital Twin predicts 130% capacity utilization, the plan must be revised. It might shift only 20% of traffic and defer the rest, or choose a configuration rollback instead.

> **The Digital Twin answers “what is likely to happen?” It does not grant permission. Passing a simulation is necessary evidence, but it is not a safety approval.**

7. Working with the Safety Engine

The Safety Engine is the enforcement layer between an AI recommendation and the real network. It evaluates whether a particular plan is permitted and bounded enough to execute.

Typical checks

• Is this target device in the automation scope?

• Is the action type allowed, and is it represented in the approved action catalog?

• Did the Digital Twin pass, including the rollback test?

• Is the blast radius within policy and is redundancy preserved?

• Is diagnosis confidence high enough for automatic execution?

• Does the plan require a human approval, maintenance window, or change ticket?

• Are verification and stop conditions present?

Possible decisions

Decision	Meaning
APPROVED_AUTOMATICALLY	The executor may run only the approved actions.
APPROVED_WITH_LIMITS	The plan may run with bounded targets, thresholds, or deadlines.
HUMAN_APPROVAL_REQUIRED	An operator must explicitly authorize the plan.

Decision	Meaning
REVISE_AND_RESUBMIT	The planner must address defined safety gaps.
REJECTED	The proposed action is not allowed or too risky.

> **A safety approval must be specific: these exact operations, on these exact targets, within these exact limits. It must never mean “the AI may do anything necessary.”**

8. Design rules that keep Aegis safe

The AI proposes; deterministic controls decide

Use the AI for interpreting evidence, retrieving relevant runbooks, forming hypotheses, and creating candidate plans. Use conventional code and policy rules for schema validation, permissions, command allowlists, risk limits, approvals, execution, timeouts, and rollback triggers.

Use constrained actions

The planner should emit an operation such as `restore_config_version(target=router-r7, version=r7-config-v41)`. A trusted adapter converts it into vendor-specific commands. The model should not emit unrestricted device CLI commands.

Make uncertainty visible

“The available evidence is not sufficient for a safe diagnosis; run these read-only checks” is a correct and valuable outcome.

Every modification needs a tested rollback

A rollback should state the state to restore, the trigger for restoring it, how success will be verified, and the safe escalation path if rollback fails.

Protect against stale plans

Before execution, re-check that the relevant configuration, topology, device health, backup capacity, and approval are still valid. A plan that was safe five minutes ago may not be safe now.

9. A practical hackathon scope

A convincing prototype does not need to support every outage. Build one end-to-end story extremely clearly:

> **A configuration change causes primary-path traffic loss. Aegis diagnoses the change, proposes a rollback or bounded traffic shift, simulates it in the Digital Twin, enforces safety policy, and executes only a specifically approved recovery.**

Demo checklist

• Ingest a JSON incident package.

• Show two or three ranked diagnoses with evidence and confidence.

• Retrieve an approved runbook or action catalog entry.

• Generate a machine-readable recovery plan with rollback and stop conditions.

• Show a Digital Twin pass or fail, then revise the plan if needed.

• Show the Safety Engine approving, limiting, rejecting, or requiring a human.

• Display the explanation, decision trail, and final verification metrics in a dashboard.

A compact module contract

Input: incident context + topology + telemetry + logs + changes + policies + action catalog + runbooks + prior feedback.
Process: build timeline -> rank hypotheses -> generate plans -> simulate -> revise -> safety review.
Output: diagnosis report + ranked candidate plans + evidence + assumptions + machine-readable actions + verification + rollback + operator explanation.

> **Central rule: your module recommends an evidence-backed, reversible recovery strategy. The Digital Twin predicts its effects, and the Safety Engine controls whether and how it may reach the real network.**